VETRA

Payments, payouts and refunds

Updated October 2026

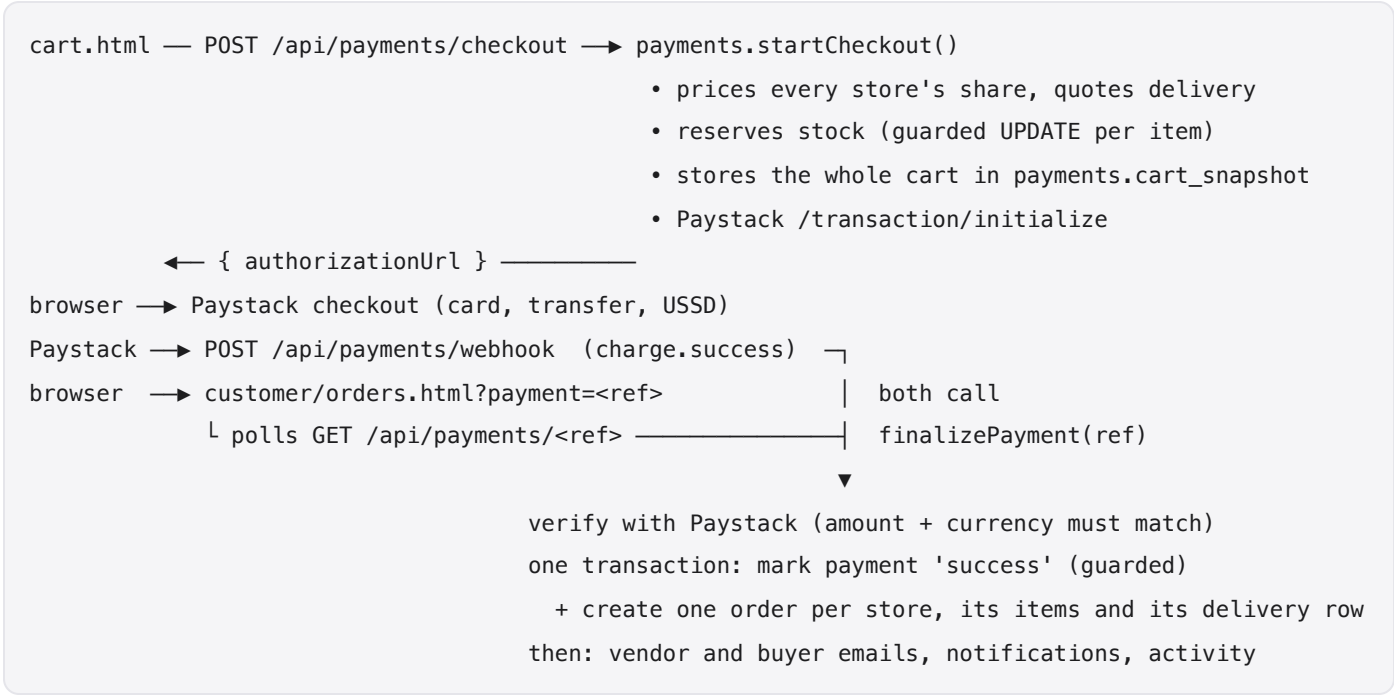
How money moves through VETRA: buyers pay through Paystack, riders and vendors are paid by Paystack transfer, and refunds go back to the buyer's card. The code is `backend/src/utils/payments.js` (all of the logic), `backend/src/utils/paystack.js` (the Paystack calls), `backend/src/routes/payments.routes.js` (checkout, the return-page check, the webhook) and migration `017_payments.sql`.

1. The money rules

	Rule
What the buyer pays	Goods + delivery for every store in the cart + a **processing fee**. The fee is grossed up so VETRA receives the full goods + delivery after Paystack's cut: 1.5% + ₦100 (the ₦100 waived under ₦2,500), capped at ₦2,000. Constants at the top of `payments.js`; `api-client.js` mirrors the formula for the cart's display.
Rider	`deliveries.rider_payout`: the zone rate × `rider_payout_percent` (80% by default), fixed when the order is placed. Paid **the moment the buyer's delivery code is entered**, and also for a parcel that's returned to the vendor. Free delivery doesn't reduce it.
Vendor	Order goods total − commission, plus the delivery fee if the vendor shipped it themselves. Paid **when escrow releases**: the buyer taps "I received it", or the protection window (48 hours by default) passes with no open report.
Commission	`platform_settings.commission_percent`, set by a Super Admin on `admin/payments.html` (0–50%, 0 by default). Taken from goods only, never delivery. Snapshotted onto each order when it's created (`orders.commission_percent`), so changing it never affects orders already placed.
VETRA keeps	Commission + (delivery fee − rider payout). The processing fee only covers Paystack.
Refunds	Goods + delivery for the affected order, to the buyer's card. **The processing fee isn't refunded** (Paystack keeps its fee on a refund), except when a payment never became an order at all, which is refunded in full.

Orders placed before payments existed have `orders.payment_id` NULL. No money was collected for them, so they never pay out or refund automatically. Riders on those jobs are paid by hand and recorded with **Mark paid** on the Deliveries page.

2. Checkout



- **No unpaid order ever exists.** Orders are only created inside `finalizePayment`, so none of the existing order and delivery queries need to filter anything out. The claim and the order inserts commit together, so a crash can't leave a payment marked paid with no orders, and the webhook and the return page racing each other create the orders exactly once.
- **Checkout requires a buyer account**, so every payment has an email. Repeating a checkout (double click, retry) with the same key returns the same Paystack link instead of starting a second payment.
- **Abandoned checkouts.** A checkout holds its stock for 30 minutes. The sweep then checks Paystack once (in case the webhook was missed). If the buyer never paid, it marks the payment `abandoned` and gives the stock back.
- **A payment that arrives too late** (after the checkout expired), **or for the wrong amount**, is never fulfilled. It's refunded in full and the buyer is emailed.
- `POST /api/orders` now answers **410**. It only exists so a cached old cart page fails loudly instead of placing an unpaid order.

---

3. Payouts

Every payout is a row in `payouts`, unique per job (`kind` + the order or delivery it's for), so no trigger can ever pay a job twice.

Status	Meaning
`queued`	Waiting: automatic payouts are switched off.

Status	Meaning
`awaiting_account`	The payee hasn't added a bank account yet. Sent the moment they do.
`processing`	Transfer started; Paystack's `transfer.success` webhook finishes it.
`success`	Money sent. For a rider, `deliveries.rider_paid_at` is set too.
`failed`	Paystack refused, or the balance was short. Retried by the sweep every 30 minutes, up to 5 attempts.
`reversed`	The bank reversed it. Treated like `failed`.

- **The master switch.** `platform_settings.auto_payouts_enabled`, **off by default**. While off, payouts are recorded but nothing leaves the Paystack balance. Turning it on (Payments page) sends everything waiting straight away.
- **No double payments on retry.** Each transfer carries a VETRA reference. Before retrying, the sweep asks Paystack about the previous attempt: if it went through, it's recorded; if it failed, the retry goes out under a fresh reference.
- **Bank accounts.** Vendors add theirs on `vendor/earnings.html`, riders on the Bank tab of the rider app. Both use `utils/payoutAccount.js`: the account name comes from the bank via Paystack, never typed; the number is AES-256-GCM encrypted and only ever shown masked. Changing the account clears the cached Paystack transfer recipient.

---

4. Refunds

Trigger	Amount
Vendor cancels an order (`PATCH /api/orders/:id/shipment`)	Everything not yet refunded
Vendor marks an item unavailable	That item; everything if it was the last item
Dispatch cancels before pickup	Everything
Parcel returned to the vendor	Everything (the rider is still paid)
Super Admin manual refund (Payments page)	Any amount up to what's left; a partial refund comes out of the goods, so it also lowers the vendor's payout
Payment too late, or wrong amount	The whole payment

- Refunds are capped atomically: `orders.refunded_amount` can never exceed goods + delivery paid for that order.

- **Refunds are never retried automatically.** Paystack's refund call takes no idempotency key, so a refund that timed out may still have gone through. It's recorded as `failed` with that warning, and a Super Admin retries it from the Payments page after checking the Paystack dashboard.
- A manual refund isn't allowed once the vendor has been paid for that order (escrow released). Recover it from the vendor outside VETRA.

---

5. Webhook

`POST https://api.vetrahq.org/api/payments/webhook`, mounted in `app.js` with `express.raw()` **before** `express.json()`, because Paystack signs the raw body (HMAC-SHA512 with the secret key, `x-paystack-signature`). A bad signature gets 401 and changes nothing.

Event	Effect
`charge.success`	`finalizePayment`
`transfer.success` / `transfer.failed` / `transfer.reversed`	Updates the payout
`refund.processed` / `refund.failed`	Updates the refund

Every handler is idempotent, and the route always answers 200 once the signature checks out, so Paystack's own retries are harmless.

---

6. The sweep

`backend/scripts/delivery-cron.js` (cPanel cron, every 10 minutes) runs the delivery sweep, then `runPaymentsCron()`:

1. expire abandoned checkouts and give their stock back;
2. while automatic payouts are on: send `queued` payouts, `awaiting_account` payouts whose payee now has an account, and `failed` payouts due a retry.

Escrow releases happen in the delivery sweep first, so the vendor payouts they create go out in the same run.

---

7. Endpoints

Route	Who	Does
`POST /api/payments/checkout`	buyer	`{ groups: [{ vendorId, items: [{ productId, quantity }] }], deliveryMethod, deliveryAddress?, deliveryState?, deliveryPhone?, deliveryNotes?, idempotencyKey }` → `{ reference, authorizationUrl }` or `{ reference, alreadyPaid }`
`GET /api/payments/:reference`	buyer (own)	Status, tracking codes once paid, `refunded` for a late payment. Finalizes if the webhook hasn't arrived.
`POST /api/payments/webhook`	Paystack	Signature-checked, see §5
`GET/PUT /api/vendors/me/payout-account`, `/banks`, `/resolve`	vendor	Bank account
`GET/PUT /api/delivery/rider/payout-account`, `/banks`, `/resolve`	rider	Bank account
`GET /api/vendors/me/earnings`	vendor	Each order's goods, commission, net and payout; totals
`GET/PATCH /api/admin/payments/settings`	Super Admin	`commissionPercent`, `autoPayoutsEnabled`
`GET /api/admin/payments`, `/payouts?status=`, `/refunds`	Super Admin	Lists (200 rows)
`POST /api/admin/payouts/:id/retry`, `/refunds/:id/retry`	Super Admin	Retry
`POST /api/admin/orders/:id/refund`	Super Admin	`{ reason, amount? }` (kobo; omit for everything). `:id` may be the tracking code.

Route	Who	Does
`GET /api/admin/paystack/balance`	Super Admin	Balance card

---

8. Data

- `payments`: one per checkout. Reference, buyer, email, goods / delivery / fee / amount (kobo), status (`initialized`, `success`, `failed`, `abandoned`), the cart snapshot (JSON), the idempotency key, Paystack's transaction id and channel, `expires_at`.
- `orders` gains `payment_id`, `commission_percent`, `commission_amount` (set at escrow release), `refunded_amount`.
- `payouts`: kind (`vendor` / `rider`), payee, order or delivery, amount, VETRA reference, Paystack transfer code, status, attempts, failure reason.
- `refunds`: order (NULL for a payment that never became one), payment, amount, reason, status, Paystack refund id, failure reason.
- `users.paystack_recipient_code`: cached transfer recipient.
- `platform_settings` gains `commission_percent` and `auto_payouts_enabled`.

---

9. Going live (Paystack dashboard)

1. Business verified and activated for live payments, with **Transfers** enabled.
2. Settings → Preferences: **turn off OTP for transfers.** Otherwise every automatic transfer stalls and fails with that reason.
3. Settings → API Keys & Webhooks: webhook URL `https://api.vetrahq.org/api/payments/webhook`.
4. **Payouts are paid from the Paystack balance.** Collected payments have to stay in it (or it has to be topped up) instead of all settling to the bank. Confirm the settlement setup with Paystack. A short balance loses nothing: payouts fail with the reason and retry.
5. Test end to end with the `sk_test_` key and Paystack's test cards, then put the `sk_live_` key in `~/vetra-api/.env` as `PAYSTACK_SECRET_KEY`, restart, and turn **Automatic payouts** on.

---

10. Tested

Against a real MySQL 8 database with migrations 001–017 applied and Paystack mocked:

- a two-store checkout charges the right amount and reserves stock;

- a repeated checkout reuses the payment;
- the webhook and the return page racing create exactly one order per store;
- wrong-amount and late payments are refunded, not fulfilled;
- expired checkouts restore stock;
- a rider is paid on the delivery code, but only once auto-payouts is on, and only once;
- the vendor is paid goods minus commission on escrow release, waiting for a bank account if needed;
- a failed transfer retries under a new reference;
- refunds are capped;
- legacy orders never pay out;
- a wrongly signed webhook is rejected;
- every money endpoint refuses non-Super Admins, buyers only see their own payments, and commission is hidden from buyers.

Not yet tested: a live Paystack test-mode run, which needs the keys and the webhook URL set (§9).